

CMPE 220- System Software

Software CPU Design

Project Report

BY

Sanjushree Golla

Gandhi Soumya Atluri

Vignesh Jetty Ravi

Under the guidance of Prof. Ishie Eswar

27 April 2026

Github Repository

<https://github.com/SANJUSHREE19/220-System-Software>

CPU Architecture Design

Overview

This project, developed in C++, implements a software-based CPU that can load and execute assembly programs written in a custom instruction format. The design is intentionally kept simple to make it easier to understand key components such as memory, registers, the ALU, the control unit, and the fetch–decode–execute cycle. While the overall structure follows a standard CPU model, it has been adapted to match our specific implementation and naming conventions.

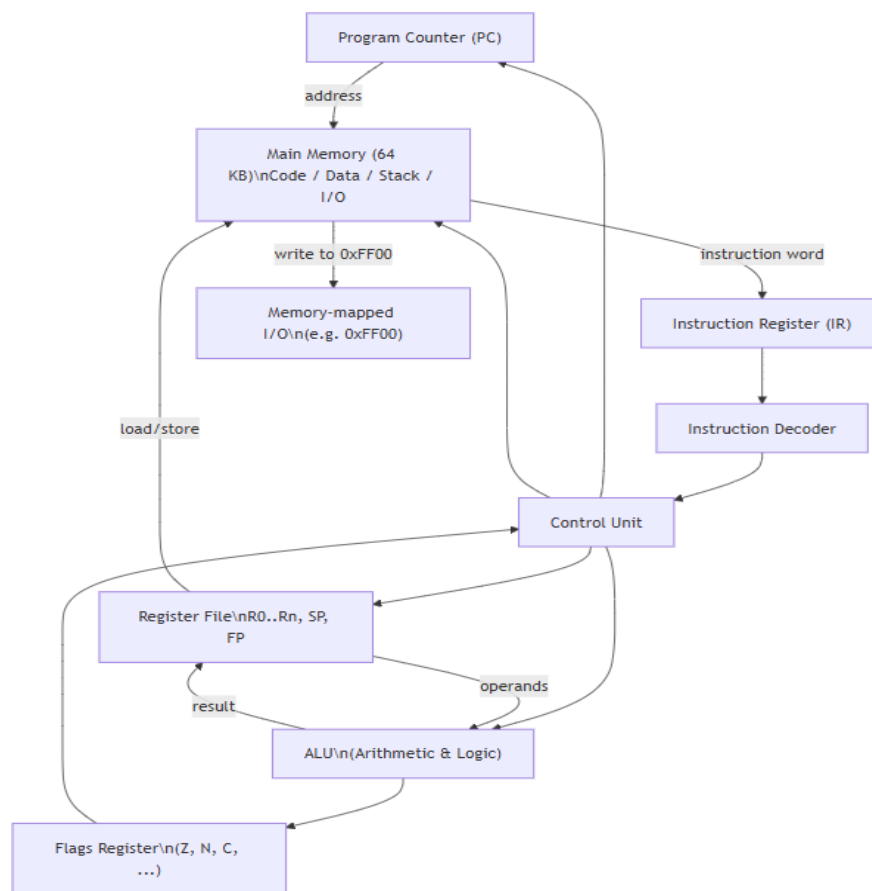


Figure 1 - CPU Architecture Design

Steps to download, compile and run the program

1. Download the project

Open the [GitHub repository link](#), in order to download the code you can do one of two things.

Option A - Download ZIP

- Click on the green **Code** button.
- Select **Download ZIP**.
- Extract or unzip the ZIP file to a folder on your computer (for example: CPU_Design).

Option B - Clone with Git

To clone from git, type the following code into terminal:

```
git clone https://github.com/SANJUSHREE19/220-System-Software
```

After this, you should see folders such as:

- In src/ - All C++ source code for the CPU, memory, assembler, and main driver.
- In asm/ - All example assembly programs (hello_world.asm, fibonacci.asm).
- README.md file with quick overview information and example commands.

2. Build (compile) the emulator

1. You need to have a C++ compiler installed (ex: g++) in order to be able to create the emulator.
2. From the root of the project folder (where src/ is located), run:

3. Windows -g++ -std=c++11 -I./src src/*.cpp -o emulator **macOS:**

clang++ -std=c++11 -I./src src/*.cpp -o emulator (or g++ if installed)

4. Using the command line shown above in Windows, an exe file named emulator.exe is created from all files found in the src/ directory and compiled to create a single executable named emulator into the same folder.

3. Run the programs and interpret the results

The emulator command-line interface is:

```
./emulator run <hello_world.asm> or ./emulator.exe run <fibonacci.asm>
```

```
./emulator step <hello_world.asm>
```

- run command - assembles an input assembly file and loads it to the memory and continues to run until a HALT instruction is encountered.
- step command- will have the assembler assemble and run each instruction in a sequential manner. Print out the status of the CPU after every step.

Below we describe what is happening in detail with the **result images** for each of the two programs.

3.1 Hello World program

Command

```
./emulator run asm/hello_world.asm
```

Program behavior

The Hello World program shows an example of a program that uses memory-mapped I/O.

- The program loads the base address where the string HELLO_MSG resides into a register.
- The program writes the address of the output port (0xFF00) into the other register.
- Within a loop, it grabs one byte of the string from memory and then checks to see if it is the 0x00 terminator and if it isn't it writes the byte value to the output port (0xFF00).
- The emulator displays what was written to the output port as a character to the console.

3.2 Fibonacci program

Command

```
./emulator run asm/fibonacci.asm
```

Program behavior

This Fibonacci program iteratively calculates the Nth Fibonacci number by:

- It loads the target required index N (7) from the data segment.
- It initializes registers for F(i-2), F(i-1), and the loop counter.
- In each loop iteration:

The loop shifts Fi-2, Fi-1 to Fi-1 and then adds the next value to the end of the current register to continue the Fibonacci (Nth) calculation, increments the Loop Counter, and repeats until the counter reaches N.

Team Contribution

Team Member	Contribution Area	Detailed Responsibilities
Sanjushree	CPU Architecture and Emulator	- Created overall CPU design that includes all of the registers, ALU design and structure, all control signals and memory configuration. - Implemented core emulator functionality (fetch–decode–execute cycle, instruction decoding, execution flow). - Developed ALU operations and flag handling (Z, N). - Implemented and validated memory-mapped I/O for output operations. - Led integration, debugging, and testing of the complete CPU system.
Gandhi Soumya	ISA Design and Assembler	- Determined Instruction Set Architecture, including the instruction formats, opcode designations, and addressing scheme. - Implemented assembler including instruction parsing, encoding, and label resolution.

		- Handled numeric literals and translation of assembly programs to machine code.
Vignesh	Programs and Testing	- Developed assembly programs (Hello World, Fibonacci, Factorial). - Performed testing and validation of CPU execution. - Verified outputs and assisted in debugging program behavior and execution traces.